

ML-Bots Final Essay

Erik Goeke

May 2026

1 Introduction

The NASA ML-Bots project focused on introducing younger students to machine learning through robotics and hands-on learning. The goal of the project was to expose as many students as possible to engineering, and to give them a chance to explore their interests in an accessible and engaging environment. This allowed them to discover whether they had an interest in STEM and gave them a path to pursue it in the future. Through programming activities and interactive demonstrations, students gained experience with industry concepts before their collegiate career.

In addition to learning technical concepts, the program emphasized creativity and problem-solving, skills students will use in life regardless of academic interests. By working through coding problems with other students, teamwork skills were also developed alongside the critical thinking and problem-solving skills that are required for learning code for the first time. The project also taught students about the use of artificial intelligence and machine learning in the real world, and discussed the ethics behind the use of AI tools. It taught students to use AI and machine learning properly, and in a way that supplements their learning.

2 ML-Bots Curriculum

The ML-Bots curriculum was designed by students at Mercer and was made in Google Colab in order to ensure it was accessible to students even when they are not in school. In the Google Colab, there are 4 separate modules.

2.1 Module 1 - Intro to Google Colab

In this module, students are introduced to Google Colab and are given learning goals for the section. This module also serves as the first introduction to Python, where students learn some basic commands and get to see some basic outputs. Examples of this include simple mathematical operations, setting variables, outputting text, and printing variables. The module then moves into hardware and shares with the students that not all hardware is built the same, so it is important to become familiar with what your device is running on. Once the students are familiar with Python and its basic abilities, they are introduced to Linux. Linux can be complex, so in this section, students are walked through each command step by step, and are taught the function of each symbol they type. They are also taught about directories and shown how a computer parses through and finds items in an interface that does not have a user interface. Once the basics of both languages have been covered, the module goes on to explain how they are connected and how they can effectively be used in tandem.

The next section in this module is "Intro to Machine Learning", which begins by covering all the useful libraries for machine learning. Specifically, numpy, Matplotlib, scipy, scikit-learn, pandas, tensorflow, and PyTorch. These libraries are all described in detail and clearly show the use case of each. With a strong background in both software and hardware, students then learn about Jetbots. Jetbots are the machine learning platform used to educate the students. They are robots run on NVIDIA's Jetson Orin chip, a chip that is used in many professional machine learning projects. The Google Colab explains how to connect the

Jetbot to the network and how to connect it to Jupyter Notebook. Students then get a chance to work with the Jetbots in an interactive, hands-on activity where they write commands to make it move.

The next section, following the interactive activity, talks about why machine learning is necessary, and gives students an exercise which highlights how machine learning can be applied to real-world problems. Specifically, this machine learning model seeks to predict whether a person survived the Titanic solely dependent upon whether the passenger was male or female. In this activity, students pull together all the skills they learned about Linux and Python, as well as the skills of finding a directory within Linux. Students also learn about the training, validating, and testing that all machine learning models go through. The module explains the differences in these processes and explains the necessity behind each part. The next part explains hyperparameters and how they characterize a machine learning model. Students then get to test their own hyperparameters and see if they can improve the model using their own values.

2.2 Module 2 - Neural Networks

The next section in the Colab covers neural networks and once again gives students learning objectives and the standards for the course. To learn about neural networks, students are provided a plethora of YouTube videos which they can access from home if they wish to re-watch any. The first subsection in this module is about regression and classification. It teaches the students how to choose between regression and classification depending on their specific scenario. The section explains that regression is used to predict continuous numerical values, such as the price of a stock or the age of a person, while classification is used to predict categorical values, such as whether a customer will buy a product or not. In order to explain regression, a plot showing hours of sleep and test scores is shown. The section explains that the regression model is fit by the standard equation of a line, which is $y=mx+b$. Linear regression is a powerful tool that allows for the understanding of the relationships behind models. However, it can't determine the probability of an event occurring. This is where logistic regression comes in, which is an example of the aforementioned classical algorithm. The module explains that logistic regression is used to predict a binary outcome, so the previous example of test scores is changed a bit to ask the question of whether a student is going to pass a test or not. The problem is now binary, meaning it either has a yes or no answer, so in this case, a classification algorithm will be used instead. The equation for this classical model is no longer as easy as $y=mx+b$, which the module explains. Now, two functions have to be used and then composed; a Desmos link is embedded in the module in order to allow students to change numbers in order to show that the large equation is not as intimidating as it looks.

The next section in this module is "Limits of Logistic Regression and Nonlinearity"; in this section, the module makes sure to explain that there are bounds to what regression can do. The biggest limitation is that regression is linear, so for models that are nonlinear, it is not applicable. The module explains that models may follow something such as a sigmoid function as opposed to a linear function, and in this scenario, linear regression can not be used to predict outcomes.

Moving on to perceptrons in the next section, the module describes that a perceptron is a type of artificial neural network that is capable of binary classification. The module describes the history of the method and explains that the perceptron is a collection of nodes, where each node represents a feature or variable input. These input variables are then multiplied by their respective weights and summed together. Images like Figure 1 are provided in the module to make this concept less intimidating.

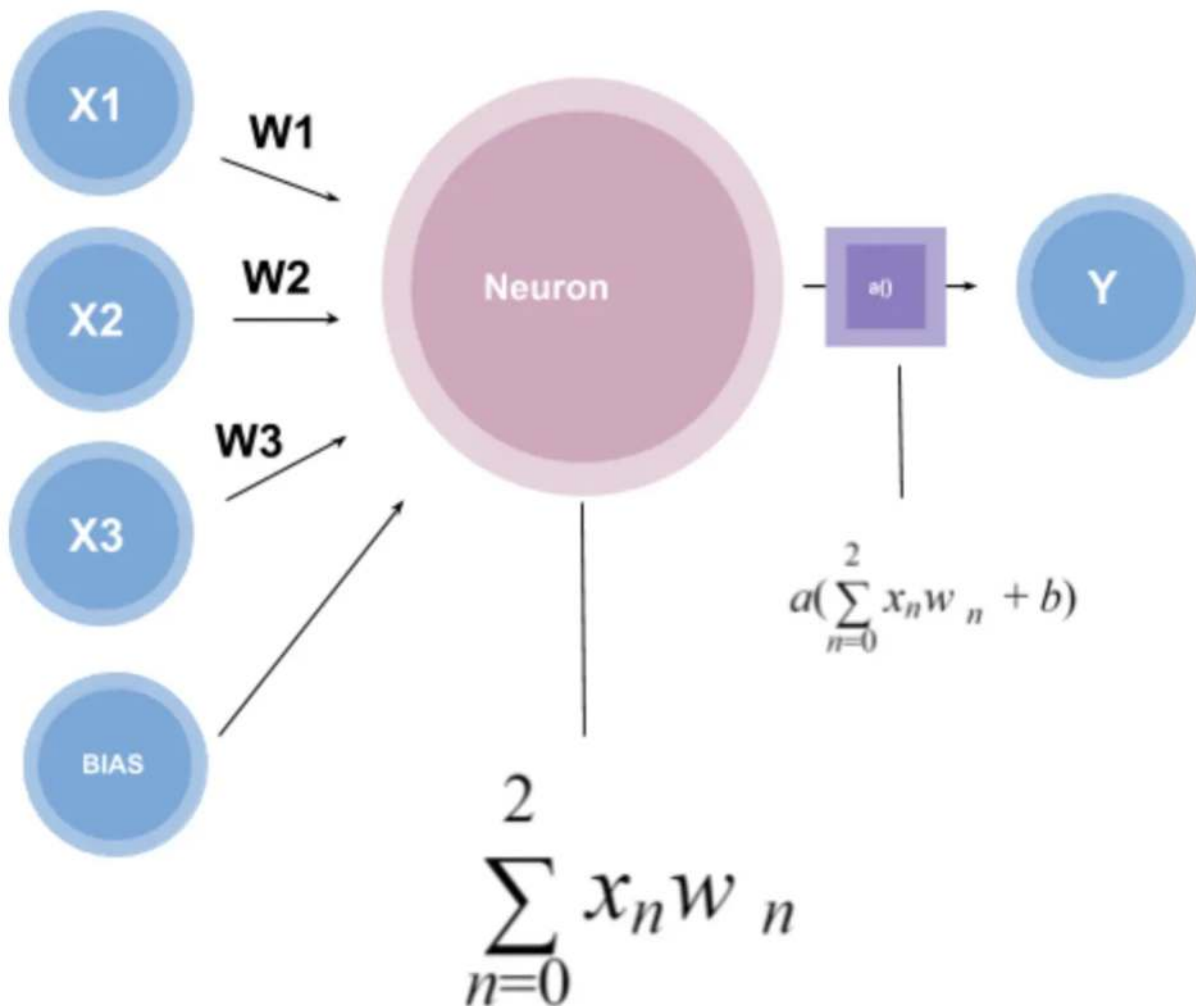


Figure 1: An easy way to visualize a perceptron model

Next, the module discusses deep learning and multilayer perceptrons, which are a more advanced concept. In order to help students visualize this, a human neural network game is provided. This game is meant to be played in student teams so that they can discuss their strategies and what is working for them. The module then moves on to describe the activation function. It explains that backpropagation is how the model learns from its mistakes, and that the general structure is:

1. The cost function outputs the loss from a training batch
2. The algorithm then calculates what changes to the weights and biases in the last layer would have moved us closer to the correct answer
3. Then we repeat step 2 for the next layer in the neural network until we are out of layers.

Say, for example, you are designing a machine model, trying to decide if something is a cat or a dog. You would need to select weights for certain values, such as a tail or a nose, and then decide how much these variables should be weighted into the decision. The model first guessed 4.2; however, you were guessing that it would guess 1 since it was a cat. This means it was over by 3.2 at the last layer, and you have to adjust weights. This means the neural network has followed its three-step process: the initial forward pass, then

figuring out how wrong it is with the cost function, and finally updating its weights and biases.

An example of automated feature extraction is provided next. Consider a scenario where you want to determine whether an employee will be promoted or not, dependent upon their performance score and their age. In this case, the linear regression would most likely not be very accurate; this is, of course, because the model that describes whether a person is promoted or not is not linear. In the module, the code is run and explained, and it determines that linear regression is 81% accurate. This is not awful, but it isn't very good either. Looking at the divide created by linear regression makes it clear as to why it does not work.

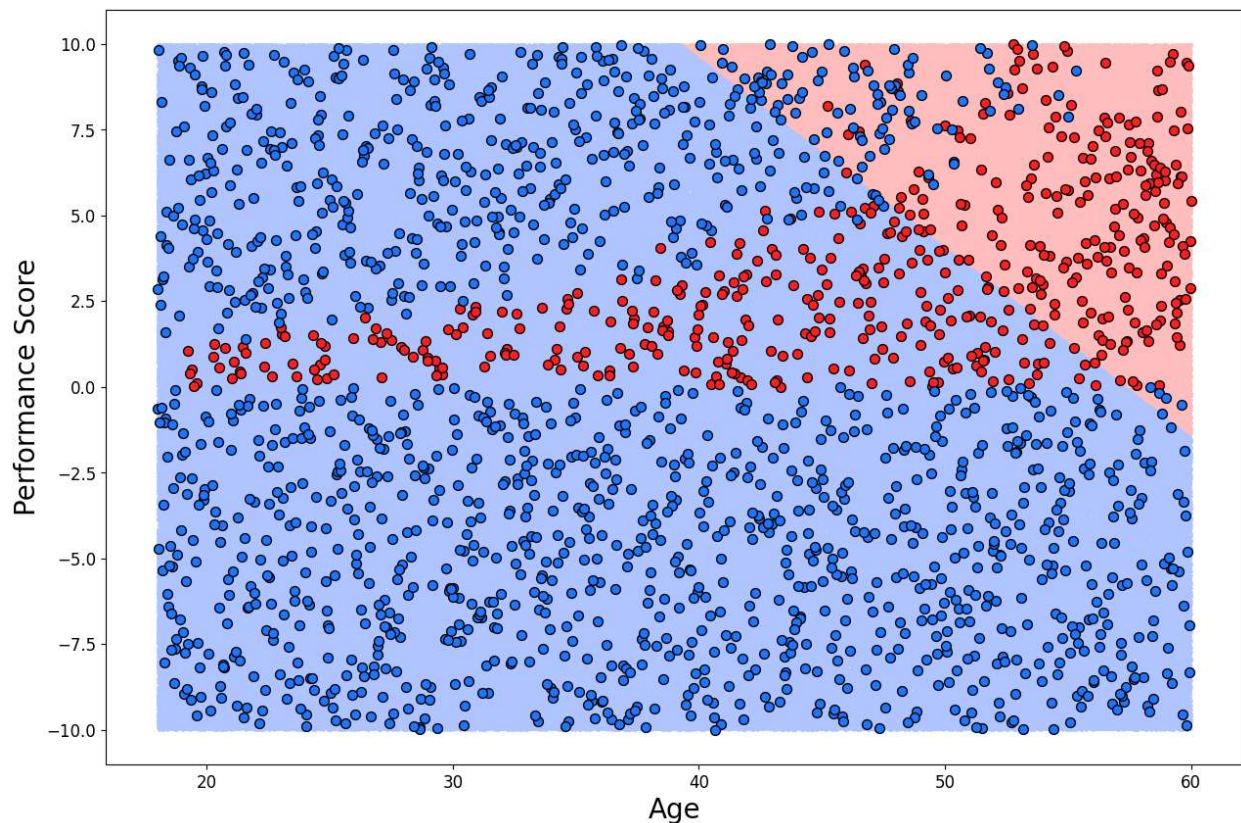


Figure 2: The divide created by a linear regression model for the promotion data

As is seen in Figure 2, the regression model is limited to creating one linear line, and therefore, it does a poor job of characterizing the data. There is a way to make regression work more effectively; however, it requires extra work. The students are then given an opportunity to work in teams in order to improve the regression model. The difficulty of engineering a solution highlights the importance of feature engineering. It allows for the data not to be linearly separable, as it only works with features, and not mathematical constants. The module then shows how easy it is for the neural network to find the same solution that took the teamwork and time of multiple students.

Once going through all these modules, the students once again get to use the Jetbots. This time, they will make the jetbot avoid obstacles and follow the road. These activities will be discussed further in another section of this paper.

The final sections in this module discuss the limitations of deep learning and explain when shallow learning should be used instead. For example, deep learning should not be used in the following scenarios:

1. When your training data is limited

2. When you are computing, the power is limited
3. When you want easier deployment
4. When interpretability is important

Finally, one last activity is provided in this section; students are given the Titanic problem from Module 1 and told to use their newly acquired neural network skills to solve the problem.

2.3 Module 3 - Genetic Algorithms

As is done in the previous modules, the start of this module describes the learning goals and provides the standard alignment.

This module begins with an intro into genetic algorithms. In this first section, the module explains that genetic algorithms take inspiration from biology and that in genetic algorithms, solutions to a problem are represented by chromosomes, much like in biology. The chromosomes are then subjected to a process of selection and recombination, much like the real-life process of natural selection. By doing this process for many generations, the model can converge on an optimal solution to the problem at hand. Genetics is the study of how traits are inherited and expressed in living organisms. Each organism has a unique set of instructions that dictate its physical and behavioral characteristics. These characteristics are then passed down to the next generation in a process of natural selection. The environment also plays a role in the expression of genetic traits, and over time, mutations occur. These mutations can be helpful, harmful, or do nothing at all, but the passing down of these traits is how evolution occurs.

Evolution also occurs in machine learning through evolutionary algorithms. These are algorithms that use the principles of natural selection and genetics to solve complex problems. For the evolutionary algorithms, there is a 5-step process to simulate the process of evolution. These steps are:

1. **Initialization:** Establishes a gene pool for the algorithm to pull from. A set of members is created with a wide variety of values depending on the context.
2. **Selection:** Members are sent through a function called the fitness function, which tests their viability for the solution. The fitness function depends strongly on the situation and expectations. The best members are sent to the next steps. This function is analogous to the process of natural selection.
3. **Crossover:** The fitness function depends strongly on the situation and expectations. The best members are sent to the next steps. This function is analogous to the process of natural selection.
4. **Mutation:** A random set of genes within individual members is changed. We do this in order to keep the gene pool wide. Without this step, our population doesn't have much room to evolve in a way that is vastly different from the parents. Essentially, without mutation, our population becomes inbred: it begins to lack the genetic diversity needed to develop radically different genetics than those of the parents' generation, which damages the algorithm's ability to explore new approaches to a problem.
5. **Termination:** We stop creating new generations once a certain number of generations that we set have been created, or we consider the solutions satisfactory.

The first step, initialization, depends on how much power can be applied to the processing. There are two common methods for initialization: random initialization and heuristic initialization. With random, a population is created with completely random individuals or values. This means that, for example, if a 10-member population, each member would have a random array of numbers assigned to them, the length of which is determined by the chromosome length. These values would fall within the preset bounds of x_1 and x_2 . On the other hand, in heuristic initialization, a general idea of the solution is already known, so the population is created using a key or rule that lacks complexity. This leads to less diversity in the population. The fitness function is also a crucial aspect of the genetic algorithm as it creates a fitness score that reflects an individual's "likelihood of survival". This score is then used to rank the remaining members

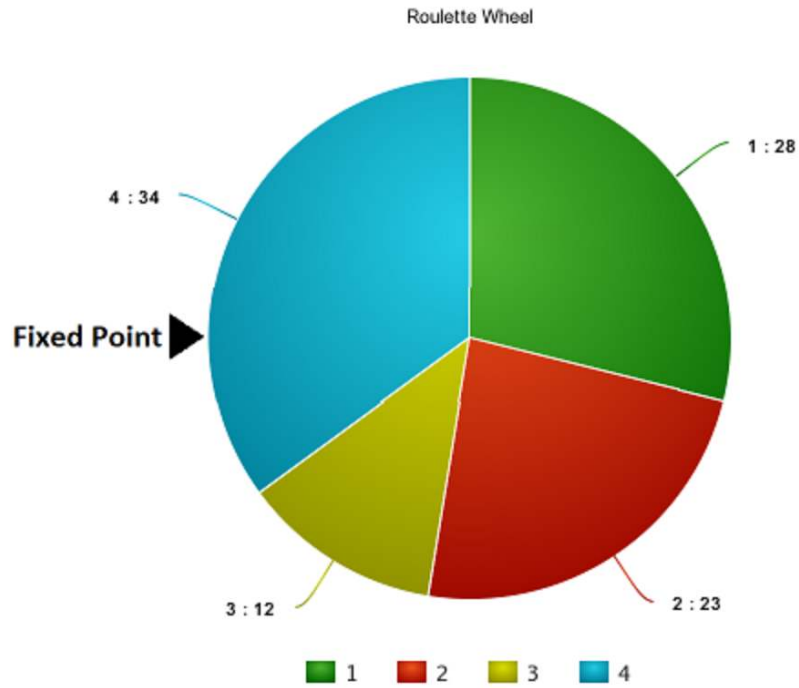


Figure 3: A visual representation of the roulette wheel selection process

of the population to find the best candidates for reproduction. Essentially, a fitness function determines how close an individual is to the desired solution.

After the fitness function has been applied to each member of the population, the next step is selecting individuals for reproduction. When choosing these individuals, the goal is to choose the most fit; however, it is also important to include diversity. One method of ensuring both of these things occur is through roulette wheel selection. The size of each wheel section is dependent upon the fitness score, and the wheel is spun twice, and the two sections it lands on are the parents; this is shown in Figure 3.

Next, the genes are crossed to find a good solution. The number of values members contain, how you prefer they learn, and any constraints imposed by the problem all determine how you will cut up the values. For example, a simple crossover may just cut the values in half and then stitch them back together, while a double crossover may cut them twice and stitch them together. This can be done as many times as you want. A single crossover is shown in Figure 4, and a double cross is shown in Figure 5.

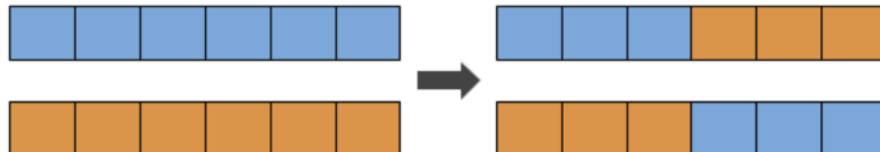


Figure 4: A visual example of a single crossover

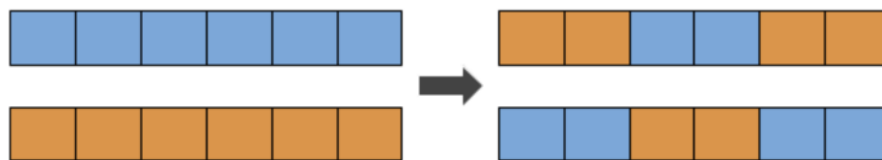


Figure 5: A visual example of a double crossover

There is also uniform crossover, which uses some process to select what genes to cross; this process may be random, or it may be dependent upon the success of individuals. An example of what this may look like is shown in Figure 6.

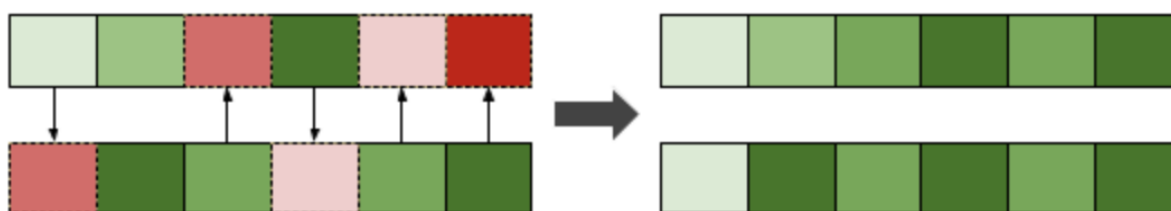


Figure 6: A visual example of a uniform crossover

Mutation is how a bit of chaos is added to the problem, and there are a few ways to do it. One method is Bit Flip Mutation, which is where random bits are chosen, and their values are inverted. Another approach is Random Resetting, which is when you replace a gene with a random value. Swap Mutation is where the position of genes is randomly swapped, and Scramble Mutation is when a selection of genes is chosen and then scrambled. Finally, inverse mutation is when a selection of genes is taken, and then their order is inverted. The final step of the process is termination, which occurs at a preset time; this can be after a certain amount of time has passed, or after several iterations.

Next, students get to play a genetic algorithm game to apply all they have learned. They also get to apply this to code in a game where they try to optimize the least amount of moves in which a mouse can get to the cheese. In order to get there in the least number of moves, the students have to modify the preset parameters. The code will output the fitness score and the average fitness of the population.

As mentioned before, genetic algorithms are a type of algorithm called an optimizer. This is how the algorithm determines how much weight to add to each characteristic. The way this is done is often through an algorithm called gradient descent. Gradients are a term learned in calculus, and they refer to derivatives where there are multiple input variables instead of the single variable that is seen most often. Students then get to try optimizing themselves by moving the sliders on the Desmos graphs in order to find minimum and maximum points. These examples all only use one variable; however, what about in a more complex problem when we have two or more? This would require extra dimensions to be added to the graph. On this graph, the optimization algorithm then seeks to find the minimum point, or a "valley". This can be visually represented by a set of balls being let out onto a surface, with the balls rolling toward an area they see as a valley. The balls all start from the same spot; however, with different "learning rates," they roll at different speeds, and thus find different "valleys". This example is visually shown in Figure 7.

The best use of a genetic algorithm is for a problem that is discrete or countable. Meaning it is a good

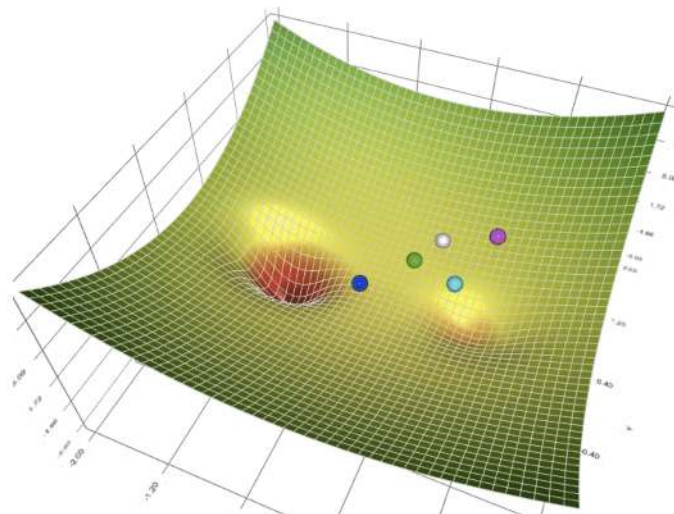


Figure 7: A visual representation of how an optimization algorithm finds a solution

idea to use a genetic algorithm to determine which cities Amazon should send its packages through. However, using a genetic algorithm to determine the ideal speed does not work because there is an infinite number of values for what speed could be. Genetic algorithms are also not ideal for pattern recognition, as they already start with an initialized population. A neural network would be a much better solution to a pattern recognition problem. Finally, genetic algorithms should not be used for problems where the optimal solution is needed. Optimizing algorithms have no way of knowing if they have found the absolute maximum or minimum. On top of this, genetic algorithms do not inherently solve for a maximum or minimum; they solve through a process of natural selection and mutation in order to get to a working solution, and this working solution is not always the best or most optimal.

Students then get the chance to try optimizing a more difficult problem, the traveling salesman problem. Essentially, in this problem, you are trying to optimize with the shortest amount of time that can be spent traveling between cities, while not visiting any city more than one time.

2.4 Module 4 - NLP

Once again, in this final module, the learning goals and standard alignment are provided.

NLP, or natural language processing, is a subset of machine learning that deals with teaching computers how to understand, interpret, and/or use human language. NLPs have made massive strides in recent years with the emergence of AI engines such as ChatGPT and Claude. However, tools such as Siri and Alexa are also NLPs and have been around for many years. The earliest attempt at NLPs was in the 1940s when researchers first explored how computers could use human language. The motivation came from World War II, where people began to realize the importance of being able to quickly translate languages. These first attempts sought to manually code in an algorithmic representation of grammar and word meaning; however, researchers soon realized this strategy was not very effective. In the 1950s and 60s, scientists developed statistical solutions to the problem; however, while better, even these solutions face major problems. Finally, after some time, in the 1980s, statistical models began to pull ahead of other models.

In the 2000s, a major development in NLP was recurrent neural networks, which were used to predict a word given its preceding words. This technology allowed for more accurate and robust language modeling, such as speech recognition, machine translation, and text classification. A few years later, a variant of RNN called Long-Short-Term Memory (LSTMs) network came out, and it was developed to overcome the RNN's struggle with long-range dependencies. However, LSTM networks still had problems with complexity and

were very prone to overfitting. Next, in the mid 2000s, sequence-to-sequence models were developed, which are neural networks that can map fixed-length input sequences to fixed-length output sequences. This helped out greatly with NLPs since the order in which words are formulated matters for meaning. Attention mechanisms soon followed and allowed neural networks to selectively focus on different parts of an input sequence during generation. Then, finally, attention mechanisms gave way to the transformer model, which is what is currently used. In the past 5-6 years, NLP technology has exploded based on transformer architecture, and new AI models such as ChatGPT are completely built on the transformer architecture.

Multi-task learning is a type of machine learning technique that involves training a neural network to perform multiple tasks simultaneously. This is advantageous because the network can easily share information as opposed to relying on individual networks. A common implementation approach to multi-task learning is transfer learning. This is when the network the model is trained on is different from the one it will be used on, with the goal being to extract useful patterns for different and more useful tasks than the main one.

One attempt at an NLP is called word embeddings, which was developed in 2003. Unlike the previous NLP attempts at NLP models, word embeddings sought to characterize the meaning of words through the context of their use. In this model, words are meaningless; only the structural system built by words matters. The question then becomes, how are you able to encode the relationship of words? The answer the researchers come up with is a graph. Say, for example, you have a plot, and the word "dog" is a point. On this plot, words that are commonly used alongside dog would appear close to the "dog" point on the graph, say for example the word "good". Words that are often not used with the word "dog", such as "river," are then located far away on the plot. However, words such as "winding" and "water" would most likely fall close to the word "river", but far from the words "dog" and "good". This can be repeated for many words, and soon a model is created. An example of the result of this process is shown in Figure 8.

Students also get to work through a similar example using the word2vec model trained on the Google News dataset. The previous example used only two dimensions, as is shown by the standard two-dimensional plot used in Figure 8; however, the word2vec model uses 300. This model is also trained on all the words and information available on Google News, again, on a much larger scale and with a more accurate example than the one previously discussed. The interesting thing about the word2vec model is that since it displays words as vectors, it is possible to do math with them, for example:

$$KING - MAN + WOMAN = QUEEN \quad (1)$$

Mathematically, it is possible to see the value that is output based on word similarity values. The closer these values are to 1, the more confident the model is in its answer. So for Equation 1 the word "king" is similar to king with a value of 1, and the word "queen" was similar with a value of 0.65, then after the math was done "queen" became similar with a value of 0.73, and "king" became similar with a value of 0.844, so while "king" was more similar, the model has moved in the direction of queen. This is shown in Listing 1. The output for this section of code is set to display the top 10 words in the word2vec dictionary that are most similar to the newly created vector. It is important to understand that the words are transformed into vectors in order to do the similarity calculation. This means that weights can be easily added to these values should the desired outcome not be achieved.

Listing 1: Word Embedding Analogy with Output

```
1 new_vector = wv['king'] - wv['man'] + wv['woman']
2 print(wv.most_similar(positive=new_vector))
3 [('king', 0.8449392318725586), ('queen', 0.7300517559051514), ('monarch',
  0.645466148853302), ('princess', 0.6156251430511475), ('crown_prince',
  0.5818676352500916), ('prince', 0.5777117609977722), ('kings',
  0.5613663792610168), ('sultan', 0.5376775860786438), ('Queen_Consort',
  0.5344247817993164), ('queens', 0.5289887189865112)]
```

In fact, in this example, if the vectors are multiplied by 2, the "queen" becomes more similar than the king does, with the queen achieving a value of 0.657 and the king a value of 0.592. This is the aforementioned

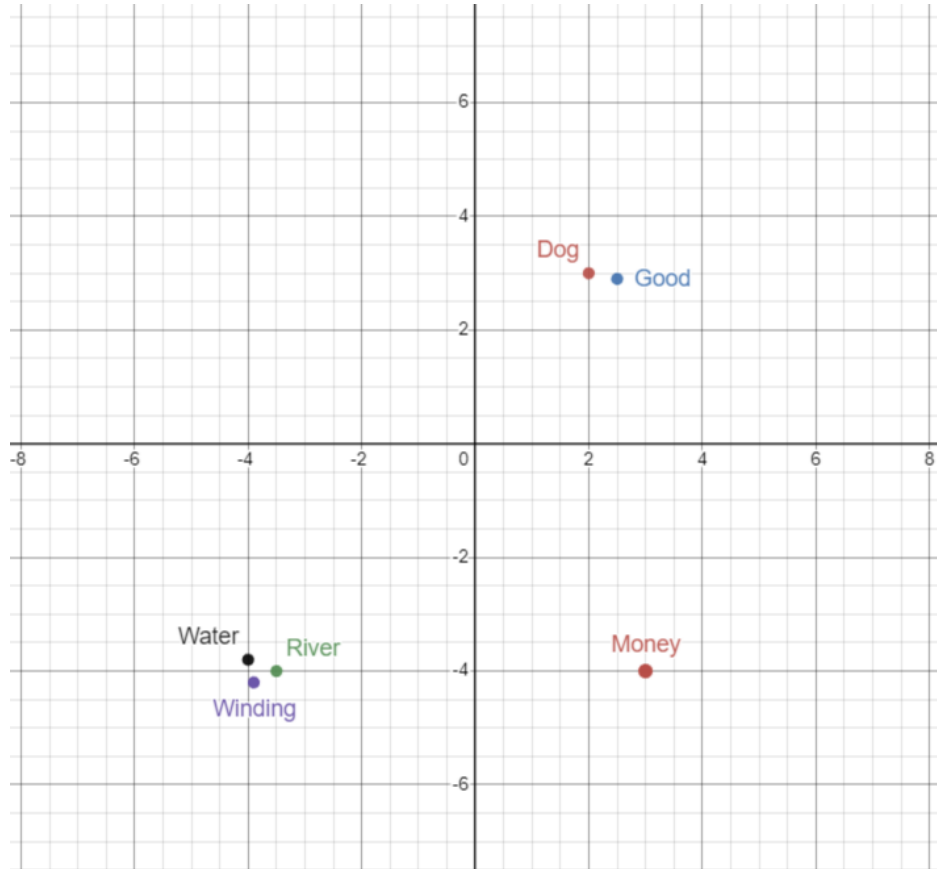


Figure 8: A plot showing how word embedding models are developed

process by which weight is added to words deemed more important in a relationship. In this way, it is interesting to see what relationships can be formed between common words and mathematical equations. This update to the code and its output is shown in Listing 2, and students are encouraged to see what other outputs they can achieve through the manipulation of weights.

Listing 2: Weighted Word2Vec Analogy Example

```
1 new_vector2 = wv['king'] - 2*wv['man'] + 2*wv['woman']
2 wv.most_similar(positive=new_vector2)
3 [('queen', 0.6566157341003418), ('king', 0.5921683311462402), ('princess',
  0.5760599970817566), ('monarch', 0.5358361601829529), ('woman',
  0.5025594830513), ('Queen_Consort', 0.4989304840564728), ('empress',
  0.4850229322910309), ('Princess_Sikhanyiso', 0.4749371409416199), ('queens',
  0.47436732053756714), ('very_pampered_McElhatton', 0.47244659066200256)]
```

In the next section, students are asked to create a game of semantle. Semantle is a spin-off game of Wordle, where you guess a word, and the output is a similarity score from Word2Vec. The code for this game is shown in Listing 3.

Listing 3: Word Similarity Guessing Game

```
1 import random
2 words = {'apple', 'banana', 'road', 'person', 'king', 'walk', 'fight'}
3 random_element = random.choice(list(words))
4 print("Hidden word is:", random_element)
5 cont = True
6 while(cont):
```

```

7  try:
8      guess = input("Guess a word!: ")
9      similarity = vw.similarity(guess, random_element)
10     if similarity == 1.0:
11         print("You guessed the word!")
12         cont = False
13     else:
14         print("Not Quite. Your word has a similarity score of:", similarity)
15 except:
16     print("That word isn't in the dictionary!")
17 The hidden word is: ki.ng.
18 Guess a word!: queen.
19 Not Quite. Your word has a similarity score of: 0.73
20 Guess a word!: king
21 You guessed the word!

```

This process seems great; however, there is a problem. What about for a word like "bank", this word could mean both a river bank and a bank used for money. This problem is called word sense disambiguation, and it was a major roadblock for NLP scientists. It wasn't until 2017 that a solution called the attention mechanism was developed. Another problem is that words have more meaning than just how they are written. This is to say, the order in which words are written matters as well. Word embeddings would not be able to distinguish what order means, so this is where seq2seq comes in. This model was developed for the growing need to output text from input text, or in other words, going from one sequence of words to another sequence of words. To understand how seq2seq works, it is useful to understand recurrent neural networks or RNNs. Recurrence is a concept in computer science where the same process is applied to data in order to break it into pieces. This is often done by passing a word through a neural network, and then looping it back onto the input so that it can be incorporated into the next input. This process is shown visually in Figure 9.

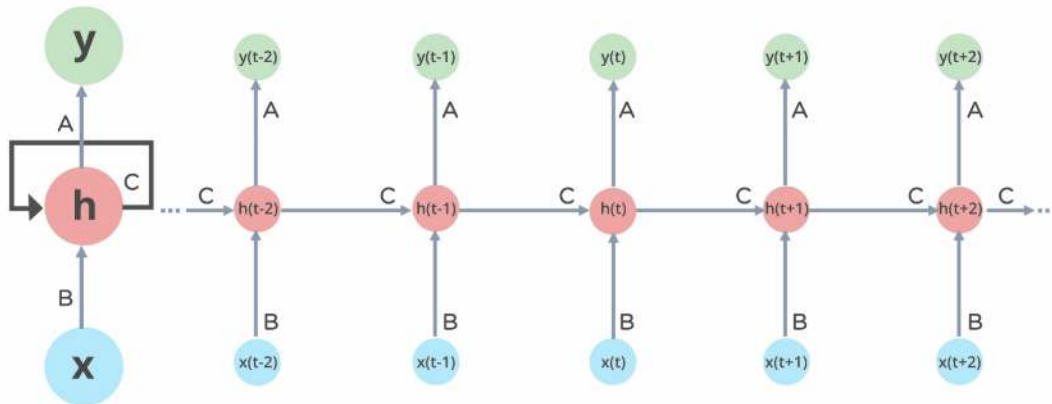


Figure 9: A visual representation of how an RNN works

The left section displays the general structure of an RNN. There is an input layer (X), a hidden dense layer (h), and then finally an output layer (y). The (h) is then fed back into the next input and then repeated for each input cascaded down towards the right side of Figure 9. It is important to note that the weights for each input remain the same; this is the key that allows for the handling of sequences that vary in size.

As it sounds, the encoder-decoder architecture is a two-step process where the first encoding step is like the word2vec model explained previously. The decoding step is the opposite of the encoder step, and it involves taking numbers and outputting words. This encoding and decoding architecture is most common in NLPs today and is the driving force behind large LLMs such as ChatGPT. However, this still doesn't

account for the NLP’s inability to capture long-range dependencies. In other words, the longer the sequence, the poorer a job the NLP does with it. This is because the beginning of the sequence is often forgotten or held in less importance by the time the NLP reaches the end of the sequence. This is where the attention mechanism comes in; it compensates by adding a weight between words in a sentence. It takes time to learn what words in a sequence matter to other words, as well as which words are less important. This is the core of what makes a transformer, say for example, let’s say you have a simple translation task. To translate a sequence, it is first fed through a group of encoders, which then in turn send the data to the decoders, which outputs the translation. An example of what an encoder would look like can be seen in Figure 10a and an example of a decoder can be seen in Figure 10b.

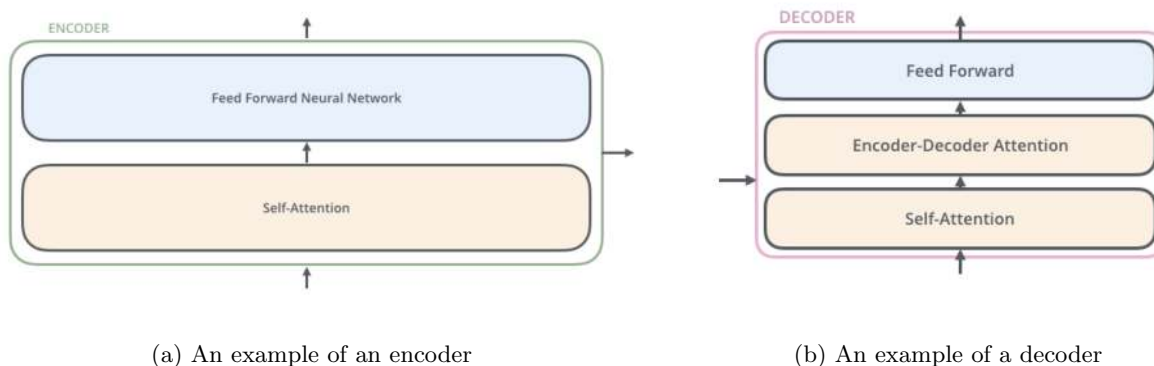


Figure 10: An encoder and a decoder represented side by side

As mentioned previously, the key part of both the encoder and the decoder is the attention layer, called self-attention in Figure 10a and 10b.

Now that the students know how NLPs work, they learn about how to do NLP. For example, how are these large models trained? So far, only models with a known solution have been discussed. This is because a labeled dataset was used; however, what about in a situation where there is no clear right answer? An example would be BERT, which runs Google’s search engine. This model was trained on Wikipedia and every unpublished book Google could find. This is a massive amount of data, and it would be impossible for a human to do on their own. To train the model, a few sequences are hidden from the model, and then it is asked to fill in the missing information using the surrounding words. This strategy allows the model to learn from all the text without requiring much human intervention. Models like this can then be fine-tuned with less data to make them more accurate.

Finally, what about adding a human to the loop? How can we go from the aforementioned transformers and make them sound like a human? This is the problem ChatGPT solved when it originally came out. The solution is simple: the model outputs text, and then humans grade it based on how much it sounds like a human. This information is then used as the next output, and the process is repeated until a human-sounding model is achieved.

3 Jetson Activity

The main activity for the students is the Jetson activity. This is when students get to operate Jetson bots that run on the NVIDIA Jetson Nano chip on their own. The first activity is basic motion; in this activity, the students learn the basics of teleoperation. The second activity is collision avoidance, which is where students first get to see machine learning in action, where they get to train their own model. Then, as a continuation to collision avoidance, students get to participate in road following.

3.1 Jetbot Setup

Before any of these activities, however, the Jetson Nano first has to be set up with the proper software. This is done through visiting NVIDIA's website, and then downloading the image file [2]. Using the Etcher software [1], flash the downloaded .img file onto the microSD card. The jetbot can then be booted, and a monitor, keyboard, and mouse can be connected using the appropriate cord. The next step is connecting the Jetbot to a Wi-Fi network, which is done through the following command shown in Listing 4.

Listing 4: Connect to WiFi

```
sudo nmcli device wifi connect <SSID> password <PASSWORD>
```

Once a network is connected and an IP address is shown on the Jetbot's LCD screen, the following steps should be taken [2]:

1. Shutdown JetBot using the command line:

```
sudo shutdown now
```

2. Unplug your HDMI monitor, USB keyboard, mouse, and power supply from the Jetson Nano.
3. Power the JetBot from the USB battery pack by plugging in the micro-USB cable.
4. Wait a bit for the JetBot to boot.
5. Check the IP address of your robot on the piOLED display screen. Enter this in place of `<jetbot_ip_address>` in the next command.
6. Navigate to `http://<jetbot_ip_address>:8888` from your desktop's web browser. You can do this from any machine on your local network.
7. Sign in using the password `jetbot`.

NVIDIA Jetson Nano has the following architecture, as shown in Figure 11.

3.2 Basic Motion

In the first basic motion module, students will log in to the Jupyter Notebook and write commands to make the JetBot move around a maze. This information must be hard-coded; there is no machine learning yet. The students soon learn just how much work goes into doing this manually, even in teams. A very basic example of this code is shown in Listing 5.

Listing 5: Basic JetBot Movement Commands

```
1 robot.left(0.3)
2 time.sleep(0.5)
3 robot.stop()
```

Students also get to make buttons on their screen to move the jetbot, as well as sliders, and see how a connection can be made between the UI on their screen and the physical jetbot in front of them.

3.3 Collision Avoidance

After the basic motion section, students are left wishing there was a simpler and easier way to navigate the maze. This is where their first introduction to machine learning comes in. In this module, the students will take images and train their own machine learning model to navigate a course. An example of a student doing this activity can be seen in Figure 12.

In this image, the student and a partner are taking images of what constitutes a free and a blocked position.

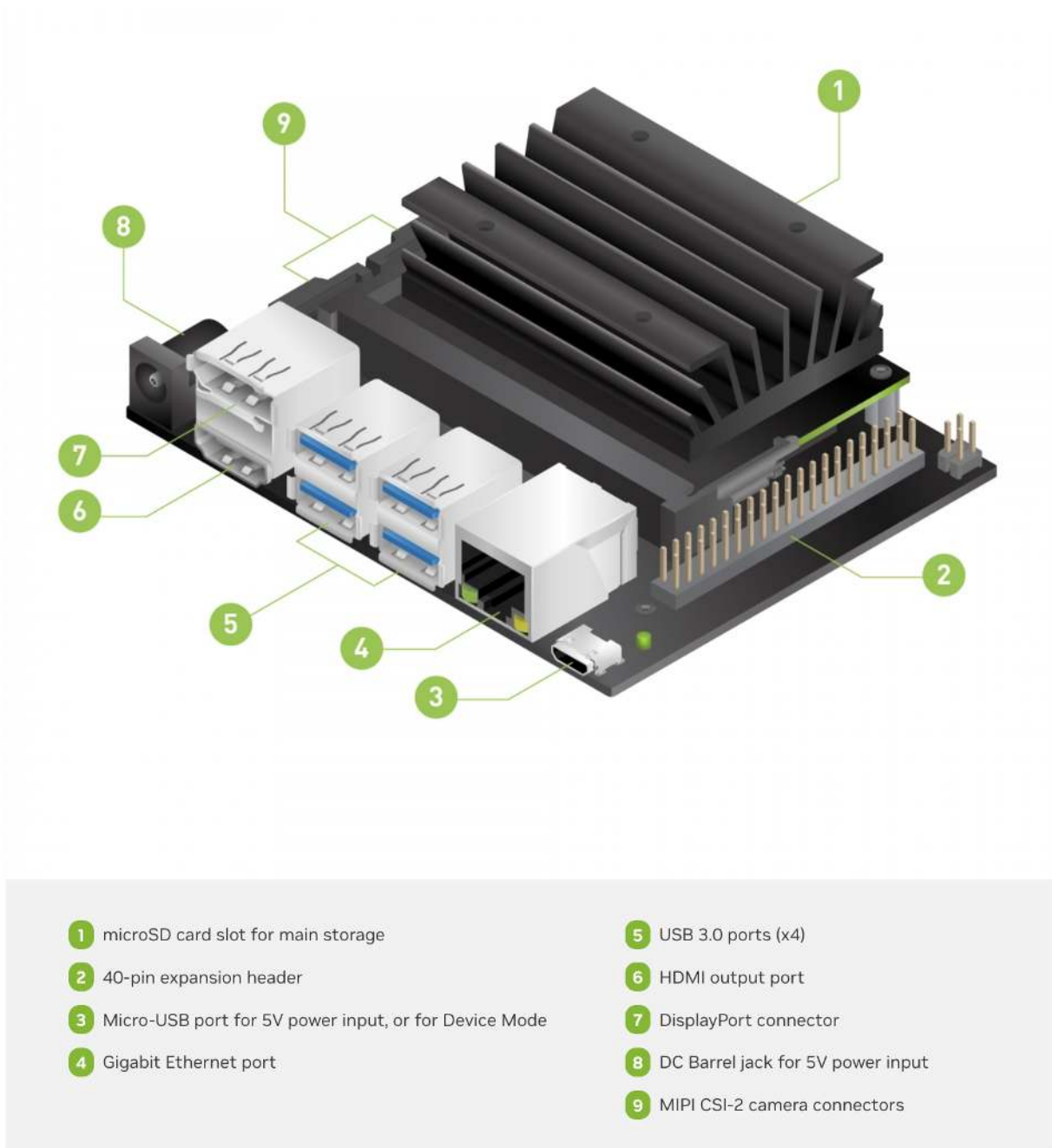


Figure 11: The architecture of the Jetson Nano chip

The students will then run this through the Jupyter Notebook algorithm and then download the .zip file to be trained. Once this occurs, a model is generated for the students with a certain percentage of certainty that it knows what values are free and blocked. Students then run this module through a module similar to the basic motion module, and see the bot traverse the course on its own, escaping blocked positions, and reaching the end with much less work than the students exerted in their attempt.



Figure 12: A student going through the collision avoidance module

3.4 Road Following

In the final module, students do the opposite of collision avoidance, and make the JetBot follow a "road" on the table. This is done similarly to the collision avoidance module, where students train a model, and then use this to make the JetBot follow a piece of tape on the table. An example of students participating in this activity is shown in Figure 13.



Figure 13: Students participating in the road following activity

4 Working with Schools

The ML-Bots curriculum and activities have been set up in schools across the southeastern United States, notably in the states of Georgia, Louisiana, Alabama, and Tennessee. High school and college academy teachers across all these states have taught the curriculum in their classrooms, and the feedback has been immensely positive.

5 Conclusion

The feedback from the mentors of this project has also been immensely positive, with some stating that it has been one of the most rewarding projects they have worked on. Allowing any student to engage in STEM and to bring it to all places in the United States is a core concept of what ML-Bots has stood for throughout its tenure. The students who have been involved in ML-Bots will no doubt continue to foster an environment of positivity around STEM among students of the younger generation. The impact ML-Bots has had on both high school students and the college mentors who have been a part of it cannot be understated. ML-Bots has by far exceeded any expectations set at the inauguration of the project, and will continue to foster positive change in all those who were involved.

All the information and resources can be found on the NASA ML-Bots website [3].

References

- [1] balenaetcher. <https://etcher.balena.io/>. Accessed: 2026-05-07.
- [2] Using sd card image - jetbot software setup. https://jetbot.org/master/software_setup/sd_card.html. Accessed : 2026 – 05 – 07.
- [3] Center for Education Integrating Science, Mathematics, and Computing (CEISMC). Nasa ml-bots, 2026. Georgia Institute of Technology. Accessed: 2026-05-07.